

RichTextEditor

Introduction

Since our release of the Froala Editor we have had a lot of positive feedback from our customers. One of the biggest suggestions we received was the ability to have images inside of the editor's content. Aside from the requests the Froala editor is not free of charge. To accommodate for these suggestions, we started working on a new editor. QuillJS is an open-source, free, editor which we abstracted in our DataFlex APIs. QuillJS is a much smaller editor in terms of its footprint. We then proceeded to add image support which works astoundingly well. There is also an experimental option for an image resizer which lets you resize your images to match the content.

Image storage of the editor

The quill editor uses the filesystem for image storage. We chose storage efficiency and speed over relational interaction. Databases have a lot of overhead compared to the filesystem due to the dynamic allocation required in a single file. Also an improvement is that the file-stream-module is not necessary and results in the application not requiring administrator privileges.

Dev include instructions

Copy the AppHtml and the AppSrc directories to your own repository. Afterwards in your index.html place the following include-snippet.

```
<!-- cWebRichTextEditor-->
<link href="RichTextEditor/quilljs/quill.snow.css" rel="stylesheet" type="text/css" />
<link href="RichTextEditor/quilljs/modules/syntax/styles/vs.css" rel="stylesheet" type="text/css" />
<link href="RichTextEditor/quilljs/modules/table-widget/hover.css" rel="stylesheet" type="text/css" />
<link href="RichTextEditor/quilljs/modules/table-widget/widget.css" rel="stylesheet" type="text/css" />
<link href="RichTextEditor/quilljs/modules/resize.css" rel="stylesheet" type="text/css" />
<script src="RichTextEditor/quilljs/modules/syntax/highlight.pack.js"></script>
<script src="RichTextEditor/quilljs/quill.min.js"></script>
<script src="RichTextEditor/quilljs/modules/table-widget/index.js"></script>
<script src="RichTextEditor/quilljs/modules/image-resize.min.js"></script>
<script src="RichTextEditor/cWebRichTextEditor.js"></script>
<script src="RichTextEditor/cWebRichTextViewer.js"></script>
```

And you are set dependency wise.

Features

The following properties are available for the editor.

- **pbAllowCodeSections:** the editor has the ability for code sections (hljs). It supports a couple of languages like JS, VB, C-, JAVA, SQL, and many others. (default: false)
- **pbAllowShowHtml:** with this function a user may add or edit the content. The editor is quite strict in its contents so of course js scripts, etc. will automatically be removed. (default: false)

- **pbUseHTMLEncoding:** for dataflex applications < 20.0 we support a minimal set of UCS2; this means that multiple languages are supported like Japanese and many others. (default: false)
- **pbFroalaTransformer:** Do you have an existing database with froala content? If this property is turned on a compatibility mode is enabled. The next time the content is saved it will have been converted. (default: false)
- **pbInlinelImages:** Indicates whether to use base64 inlining of images as opposed to uploading them to the server; use at your own risk due to the extreme size of these blobs. The server might cut them off if the field is too small.
- **puiUploadLimit:** The limit for your uploaded images. (default: 5 megabytes)
- **pblImageResizer:** The editor has a plugin for resizing images inside of the editor. Though it may be unstable in some visual situations.
- **psPlaceholder:** The editor could provide a placeholder if the content is empty.
- **psSizes:** A comma separated list of all sizes.
- **psFonts:** A comma separated list of all fonts.
- **Text (Property): (Get)** Performs a WebGet to retrieve text and **(Set)** performs a WebSet including clearing of the History and ImageChanges.
- **pbAllowTables:** Enables table editing support. (default: false)
- **piTableColumns:** Configures the number of columns shown by the table picker. (default: 5)
- **piTableRows:** Configures the number of rows shown by the table picker. (default: 6)

Also, there is a viewer included, the following properties are shared.

- **pbAllowImages:** whether you want the user to be able to upload and load images into the editor. (default: false)
- **psDirectoryName:** The name of the directory in the Data folder where the images will be saved. If the directory does not exist it will be created. (default: cWebRichTextEditor)

The following upload properties are available for the editor.

- **psUploadDir:** Upload directory used by image handling.
- **psTemporaryDir:** Temporary upload directory used before commit.
- **pbTemporaryDir:** Enables use of a temporary upload directory. (default: true)
- **pullMaxFileSize:** Maximum server-side upload size. (default: 5242880 bytes)
- **pullMaxFileNameLength:** Maximum file name length. (default: 4096)
- **pbAutoCommit:** Commits uploads immediately when enabled. (default: false)

The following helper- procedure and functions are available for the editor.

- **(Fn) RequestImages (sHtml : String) -> String:** This function is provided in the base Mixin, it allows given the output html from the editor to be processed and insert all images at their corresponding locations.
- **(Fn) HasImage () -> Boolean:** Returns whether the current content on the client holds an image.
- **(Fn) HasChanged () -> Boolean:** Returns whether the editor content has changed.
- **(Fn) SelStart () -> Integer:** Returns the beginning position of the current selection.
- **(Fn) SelEnd () -> Integer:** Returns the ending position of the current selection.
- **(Fn) Selection () -> String:** Returns the currently selected text.
- **(Ps) SetSelection (iStartSelection : Integer, iEndSelection : Integer) -> Void:** Sets the client selection range.

- **(Ps) SaveImages () -> Void:** Results in the client's image stack being cleared all image operations will be pushed to the server. This involves sending all image data that was allowed by the **OnNewRegisteredImage** event.
- **(Ps) DeleteImages (sHtml : String) -> Void:** Results in the images found the current psValue or passed sHtml to be deleted this should be called before a Request_Delete.
- **(Ps) Clear () -> Void:** Clears the editor, clearing the history (e.g. cntrl-z), clearing the image stack discarding all operations.
- **(Ps) ClearImageChanges () -> Void:** Clears clearing the image stack discarding all operations.
- **(Ps) ClearHistory () -> Void:** Clears the history (e.g. cntrl-z).

The following events are available for the editor.

- **(Fn) OnNewRegisteredImage (sFilePath : String) -> Boolean:** This function is called when an image would be added to the editor on the client. If this function returns false the adding is canceled involving no message (should be provided by the programmer). If true the image will be registered but not yet uploaded, this happens when the **SaveImages** procedure is called.
- **(Ps) OnFileFinished (sFileName : String, sLocalPath : String) -> Void:** Called when an upload finishes successfully.
- **(Ps) OnFileFailed (sFileName : String, sLocalPath : String, iErrorCode : Integer, sErrDetail : String) -> Void:** Called when an upload fails.
- **(Ps) OnSelectionChanged () -> Void:** Called when selection tracking is enabled and the selection changes.

For basic usage we would like to refer you to <https://learning.dataaccess.com/>.